

DTU Grocery Price Compare - Assignment

The Great DTU Grocery Race

A build challenge for CarDekho Group campus applicants

The problem

Every hostel resident at DTU has had this exact argument at 11 PM: *“Is this cheaper on Blinkit or Instamart?”* Right now, answering that means opening two apps, searching twice, and squinting at prices while your snacks (hopefully) stay in stock.

Your job: **build a tiny web app that settles the argument in one search.**

A user types an item — say, “Maggi” or “Amul butter” — and your app shows matching listings and prices from both Blinkit and Instamart, for delivery to DTU. That’s it. No logins, no mobile app, no reverse-engineering anything — everything you need is sitting right there on the desktop websites.

We’re not grading you on how polished the UI looks. We’re grading you on how you think: how you scope a real, messy problem, the calls you make when the data doesn’t line up cleanly, and how clearly you can explain those calls to someone else.

What you’re building

A web app where a user can:

- Search for a grocery/food item
- See matching listings and prices from **both** Blinkit and Instamart
- Get results relevant to delivery at **DTU**

Any language, framework, or AI coding tool is fair game. Use whatever gets you to a working, well-reasoned solution fastest.

What to submit

1. Your code, zipped Exclude `node_modules`, virtual environments, and build folders — we want to see what you wrote, not what npm installed for you. Include a **README** with setup and run steps that work on a fresh machine (assume we’ve never touched your stack before).

2. A one-page design note (inside the zip) This is the part we read most closely. Cover:

- Your overall architecture, and *why* you chose it over the

- alternatives you considered
- How your app decides a listing on Blinkit is “the same product” as a listing on Instamart — and honestly, where that logic breaks down
- What would need to change to support many locations or many more users

Keep it to one page. We’d rather read one sharp page than five padded ones.

3. A 1-minute screen recording Show the app working, end to end. Share it as an **unlisted link** (YouTube/Drive/Loom — your call).

Ground rules

- Keep your request volume to the sites reasonable — no need to hammer them
- Don’t deploy or share your app outside this assignment
- No login flows, no native mobile app, no reverse-engineering required or expected

Timeline

Submit by Friday, 11:00 PM. Late entries won’t be reviewed, so budget time for the design note — it’s easy to leave for last and rush.

Submit here

Fill out the response form with your details, code zip link, and recording link before the deadline:

<https://forms.gle/yico2VzLy8G6f7jL6>